

Seam Carving

Student Name: Adam FATNASSI

Report of the project for the Graph Theory Course
Master BDIA 2025-2026

Advisor
Prof. Brahim Hnich

Université Paris Dauphine
Campus Tunis

Abstract

This project implements content-aware image scaling using seam carving, a graph theory-based approach to intelligent image resizing. Unlike traditional methods that uniformly stretch or crop images, seam carving selectively removes low-energy vertical paths while preserving important visual content. We formulate the problem as finding the shortest path in a Directed Acyclic Graph (DAG), where pixels are vertices and edge weights represent gradient energy from the Sobel filter. Using dynamic programming, we achieve $O(H \times W)$ time complexity for finding optimal seams. The algorithm effectively preserves salient features like faces and buildings while removing less important regions, showcasing practical applications of graph algorithms in computer vision.

Keywords: Seam Carving, DAG, Dynamic Programming, Image Scaling, Graph Theory

Contents

Abstract	i
Table of Contents	iii
1 Introduction	1
1.1 Problem Statement	1
1.2 Seam Carving: A Graph Theory Solution	1
1.3 Real-World Applications	2
1.4 Project Objectives	2
2 Background	3
2.1 Seam Carving	3
2.1.1 Definition of a Seam	3
2.1.2 Energy Map Construction	4
2.1.3 The Seam Carving Process	6
2.2 A Graph-Theoretic Formulation	6
2.2.1 Graph-Theoretic Preliminaries	6
2.2.2 The Seam Graph	7
2.2.3 Structural Properties	8
2.2.4 Seams Are Paths	8
2.3 The Shortest-Path Formulation	9
2.3.1 Optimal Substructure and the Recurrence	9
2.3.2 Algorithms	9
2.3.3 Correctness and Complexity	10
2.4 Iterated Removal, Extensions and Limitations	11
3 Implementation	13
3.1 Overview	13
3.2 Problem Formulation	14
3.3 Dynamic Programming Approach	14

3.3.1	Dynamic Programming Table	14
3.3.2	Initialization	15
3.3.3	Recursion Relation	15
3.3.4	Seam Extraction (Backtracking)	15
3.4	Vectorised Implementation	16
3.5	Seam Removal and Iterated Carving	17
3.5.1	Seam Removal	17
3.5.2	Tracking Seams in Original Coordinates	18
3.6	Step-by-Step Example	18
3.7	Complexity of the Implementation	22
3.8	Verification	23
3.9	Implementation Notes	24
4	Conclusion	25

Chapter 1

Introduction

1.1 Problem Statement

In today’s digital world, images must be displayed on devices with vastly different screen sizes and aspect ratios—from smartphones to desktop monitors, tablets to smartwatches. The traditional approach to adapting images to different dimensions involves either uniform scaling (stretching or shrinking) or cropping. However, both methods have significant drawbacks:

Uniform scaling distorts the image, changing aspect ratios and making objects appear unnaturally stretched or compressed. Cropping removes portions of the image entirely, potentially eliminating important content. Neither method considers the actual content of the image when deciding what to preserve or remove.

Consider a landscape photograph containing a person standing in front of a castle. If we need to reduce the image width to fit a mobile screen, traditional scaling would make the person appear wider, while cropping might cut them out entirely. What we really want is to preserve the important content (the person and the castle) while removing less significant portions (empty sky or uniform background). This is the fundamental challenge that content-aware image resizing addresses: how can we intelligently resize images while preserving their most important visual features?

1.2 Seam Carving: A Graph Theory Solution

Seam carving, introduced by Avidan and Shamir in 2007, provides an elegant solution to content-aware resizing by formulating it as a graph theory problem. Instead of uniformly scaling the entire image, the algorithm identifies and removes “seams”—connected paths of low-importance pixels that run through the image from top to bottom (or left to right). A vertical seam is defined as a connected path of pixels, one pixel per row, where each pixel is adjacent to the pixels above and below it. By repeatedly finding and removing the “least important” seam, we can reduce image width while preserving salient features. The key insight is that finding the optimal seam is equivalent to finding the shortest path in a Directed Acyclic Graph (DAG), a fundamental problem

in graph theory that can be solved efficiently using dynamic programming.

1.3 Real-World Applications

Seam carving and content-aware scaling have numerous practical applications:

Responsive Web Design: Automatically adapting images to different screen sizes without manual cropping

Photo Editing Software: Tools like Adobe Photoshop implement content-aware scaling for professional image manipulation

Mobile Applications: Optimizing images for various mobile devices with different aspect ratios

Video Retargeting: Adapting video content for different display formats

Digital Art: Creating artistic effects by selectively removing or inserting content
Document Layout: Automatically fitting images into constrained spaces in publications

1.4 Project Objectives

The primary objectives of this project are:

- Implement a complete seam carving algorithm using graph theory principles
- Model the image resizing problem as a shortest path problem in a DAG
- Apply dynamic programming techniques to efficiently find optimal seams

Chapter 2

Background

This chapter introduces seam carving and shows that its central computational problem, finding an optimal seam, is a shortest-path problem on a directed acyclic graph (DAG). Section 2.1 explains the technique itself: seams, pixel energy and the iterative carving procedure. Section 2.2 builds the graph model and proves its structural properties. Section 2.3 derives the shortest-path algorithm, proves its correctness and analyses its complexity. Section 2.4 discusses iterated removal, its limits, and extensions that fit naturally into the same graph framework.

2.1 Seam Carving

Displaying an image at a different aspect ratio requires *retargeting*. The two classical operators are both unsatisfactory: *cropping* discards content at the borders, and *uniform scaling* distorts every object, whether or not it is visually important. Seam carving, introduced by Avidan and Shamir [1], is a *content-aware* alternative that changes the image dimensions by removing (or inserting) pixels along optimal paths called *seams*. It is non-uniform: different amounts of pixels are removed from different regions, according to how important these regions are. Homogeneous areas such as sky or grass absorb most of the reduction, while edges and object boundaries are largely preserved.

2.1.1 Definition of a Seam

Let I be an image with H rows and W columns. A pixel is denoted (i, j) , with row $i \in \{0, \dots, H-1\}$ and column $j \in \{0, \dots, W-1\}$.

[Vertical seam] A *vertical seam* is a sequence of pixels

$$\sigma = ((0, j_0), (1, j_1), \dots, (H-1, j_{H-1}))$$

containing exactly one pixel per row and satisfying the *connectivity constraint*

$$|j_i - j_{i-1}| \leq 1 \quad \text{for } i = 1, \dots, H-1. \quad (2.1)$$

Constraint (2.1) makes the seam an 8-connected path from the top to the bottom row. It is what prevents the removal from tearing the image: without it, one could simply delete the least important pixel in every row, and neighbouring rows would be shifted by unrelated amounts. Removing a vertical seam reduces the width by exactly one pixel; *horizontal seams* are defined symmetrically and are obtained by transposing the image.

Given a per-pixel *energy* $e(i, j) \geq 0$ (Section 2.1.2), the *cost* of a seam is

$$E(\sigma) = \sum_{i=0}^{H-1} e(i, j_i), \quad (2.2)$$

and the optimisation problem is to find $\sigma^* \in \arg \min_{\sigma} E(\sigma)$.

Size of the search space. A seam is fixed by its starting column (W choices) and by $H - 1$ steps in $\{-1, 0, +1\}$. The number of seams $N(H, W)$ therefore satisfies $N(H, W) \leq W \cdot 3^{H-1}$. Moreover, for $W \geq 2$ a seam may freely alternate between two adjacent columns, so $N(H, W) \geq 2^{H-1}$ and the number of seams is exponential in H . For an image of size $H = 542$, $W = 800$ the upper bound is of the order of 10^{261} and the lower bound exceeds 10^{163} , far beyond the reach of exhaustive enumeration. That the problem is nevertheless solvable in time linear in the image size is the main point of this chapter.

2.1.2 Energy Map Construction

To decide which seam can be removed with the least visual distortion, the importance of every pixel must be quantified. This importance is the *energy* of the pixel. High-energy pixels belong to visually significant regions such as edges, textures and object boundaries; low-energy pixels typically belong to homogeneous areas such as sky or smooth backgrounds.

Greyscale

The first step converts the RGB image into a single-channel intensity image by a weighted sum that reflects human brightness perception:

$$I_{\text{gray}} = w_R R + w_G G + w_B B. \quad (2.3)$$

The classical ITU-R BT.601 weights are (0.299, 0.587, 0.114). Our implementation uses the `rgb2gray` function of `scikit-image` [7], which uses the BT.709 weights (0.2125, 0.7154, 0.0721). Both choices give visually almost identical results, and the choice has no bearing on the graph-theoretic analysis below.



Figure 2.1: Original image and its greyscale conversion.

Energy map

The Sobel operator [6] estimates the image gradient by convolving I_{gray} with two 3×3 kernels that combine a central difference in one direction with smoothing in the other:

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}.$$

Let $G_x = K_x * I_{\text{gray}}$ and $G_y = K_y * I_{\text{gray}}$ denote the horizontal and vertical gradient estimates. Both are large at edges and textured regions and close to zero in flat regions. We define the energy of a pixel (i, j) as the gradient magnitude

$$e(i, j) = \sqrt{G_x(i, j)^2 + G_y(i, j)^2}. \quad (2.4)$$

Avidan and Shamir [1] use the ℓ_1 variant $e(i, j) = |G_x(i, j)| + |G_y(i, j)|$; the ℓ_2 magnitude (2.4) is the one implemented by common libraries and used in this work. The two behave very similarly. Moreover, the set of minimum-cost seams is invariant under multiplying e by a positive constant, so the normalisation constant of the Sobel operator (scikit-image divides by a fixed factor) is immaterial. The seam algorithm depends on e *only* through its values, which is why any other importance measure (saliency, entropy, face detectors, user masks) can be used in its place.

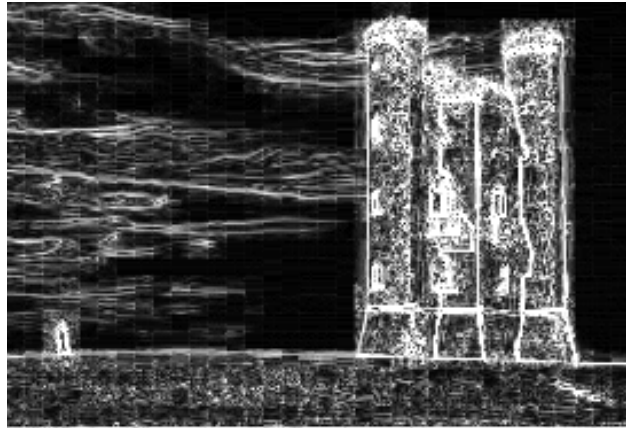


Figure 2.2: Energy map obtained with the Sobel filter. Bright pixels (edges) are expensive to remove; dark pixels (flat regions) are cheap.

2.1.3 The Seam Carving Process

To reduce the width of an image by k pixels, the algorithm proceeds as follows.

1. **Compute the energy map** e of the current image (Section 2.1.2).
2. **Find the minimum seam**: the connected top-to-bottom path of lowest total energy (2.2). This is the computationally interesting step, treated in Sections 2.2 and 2.3.
3. **Remove the seam**: delete its pixels and shift the remaining pixels of each row to close the gap; the width decreases by one.
4. **Repeat** from step 1 until the desired width has been reached.

The energy map must be *recomputed* after every removal, because deleting a seam makes previously non-adjacent pixels neighbours and thereby changes the local gradients. By repeatedly removing low-energy seams, the algorithm preserves high-energy regions (the important content) while eliminating the least important pixels. Enlarging an image works analogously: the k cheapest seams are identified and then duplicated (with averaged colours) rather than deleted [1].



Figure 2.3: Result of applying seam carving.

2.2 A Graph-Theoretic Formulation

Step 2 of the process asks for a minimum over exponentially many candidates. The key observation is that this is not a generic combinatorial search: the candidates are exactly the paths of a graph with very special structure, and shortest-path theory applies.

2.2.1 Graph-Theoretic Preliminaries

A *directed graph* is a pair $G = (V, E)$ with $E \subseteq V \times V$, together with a *weight function* $w : E \rightarrow \mathbb{R}$. A *path* from u to v is a sequence of vertices $u = v_0, v_1, \dots, v_m = v$ with $(v_{r-1}, v_r) \in E$ for all r . Its *weight* is $\sum_{r=1}^m w(v_{r-1}, v_r)$. The *shortest-path distance* $\delta(u, v)$ is the minimum weight of a path

from u to v , and $+\infty$ if there is none. A directed graph is a *directed acyclic graph* (DAG) if it contains no directed cycle. Equivalently, it has a *topological order*: a linear ordering of V in which every edge (u, v) has u before v [3].

2.2.2 The Seam Graph

Given an energy map e of size $H \times W$, we define the *seam graph* $G_e = (V, E, w)$ as follows.

Vertices. One vertex per pixel, plus a virtual source s and a virtual sink t :

$$V = \{s, t\} \cup \{(i, j) : 0 \leq i < H, 0 \leq j < W\}, \quad |V| = HW + 2.$$

Edges. Each pixel has directed edges to the (up to) three pixels immediately below it, and the source and sink are attached to the first and last rows:

$$\begin{aligned} E_{\text{src}} &= \{(s, (0, j)) : 0 \leq j < W\}, \\ E_{\text{grid}} &= \{((i, j), (i + 1, j + d)) : 0 \leq i < H - 1, d \in \{-1, 0, 1\}, \\ &\quad 0 \leq j + d < W\}, \\ E_{\text{sink}} &= \{((H - 1, j), t) : 0 \leq j < W\}, \end{aligned}$$

and $E = E_{\text{src}} \cup E_{\text{grid}} \cup E_{\text{sink}}$.

Weights. Energy is naturally a *vertex* weight. To obtain the standard edge-weighted setting, the energy of a pixel is charged to its *incoming* edges:

$$w(u, v) = \begin{cases} e(v) & \text{if } v \text{ is a pixel,} \\ 0 & \text{if } v = t. \end{cases} \quad (2.5)$$

With this convention, the weight of an s - t path equals the sum of the energies of all pixels it visits.

Size. Each of the $H - 1$ transitions between consecutive rows contributes $3W - 2$ edges (three per pixel, minus one at each of the two lateral borders), and the source and sink contribute W edges each. Hence

$$|E| = (H - 1)(3W - 2) + 2W = \Theta(HW). \quad (2.6)$$

In particular, the graph is *sparse*: the number of edges is linear in the number of pixels, and the graph never needs to be stored explicitly, since the out-neighbours of (i, j) are given by index arithmetic.

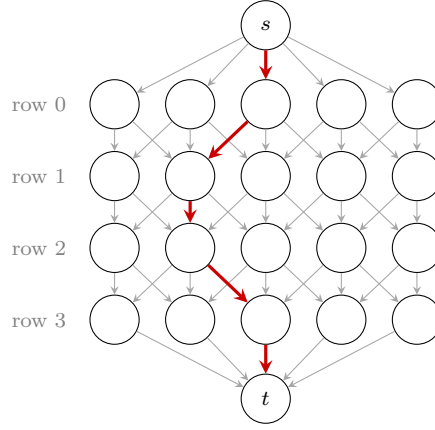


Figure 2.4: The seam graph G_e for $H = 4$, $W = 5$. Every edge points to the next row, so the graph is acyclic (Lemma 2.2.3). The highlighted path is an s - t path and hence a vertical seam (Lemma 2.2.4).

2.2.3 Structural Properties

[Acyclicity and topological order] G_e is a DAG. Ordering the vertices by row (with s first and t last, and ties broken arbitrarily) is a topological order. Assign each vertex a *level*: $\ell(s) = -1$, $\ell(i, j) = i$ and $\ell(t) = H$. Every edge $(u, v) \in E$ satisfies $\ell(v) = \ell(u) + 1$. Along any directed path the level therefore strictly increases, so no vertex can be visited twice and there is no directed cycle. Since every edge goes from a lower to a higher level, ordering vertices by level places every edge in forward direction.

The structural properties that make the graph easy to work with can now be stated precisely. The graph is *directed* (edges only go downward), *acyclic* (Lemma 2.2.3), and *layered*: the vertex set splits into $H + 2$ levels, and edges only connect consecutive levels. The topological order is available for free, without running a topological sorting algorithm, which is what will make the shortest-path computation so cheap.

2.2.4 Seams Are Paths

[Seams and s - t paths] The map

$$\sigma \mapsto (s, (0, j_0), (1, j_1), \dots, (H-1, j_{H-1}), t)$$

is a bijection between vertical seams (Definition 2.1.1) and s - t paths in G_e . The weight of the path equals the seam cost $E(\sigma)$. Let σ be a seam. By (2.1), consecutive pixels (i, j_i) and $(i+1, j_{i+1})$ are joined by an edge of E_{grid} , and the first and last steps are edges of E_{src} and E_{sink} ; so the image of σ is an s - t path.

Conversely, consider any s - t path. Its first edge leads to level 0, and every subsequent edge raises the level by exactly one; the path can only end at t (level H), so it visits *exactly one vertex in each row*. Since a grid edge changes the column by at most one, the sequence of visited columns satisfies (2.1), i.e. it is a seam. The two maps are inverse to each other. Finally, by (2.5) the weight

of the path is $\sum_i e(i, j_i) + 0 = E(\sigma)$.

$\min_{\sigma} E(\sigma) = \delta(s, t)$, and the minimum-energy seam is the shortest s - t path in G_e .

Two remarks are in order. First, the virtual source and sink are what allow us to speak of a *single-source* problem: “the shortest path from any vertex of the top row to any vertex of the bottom row” is a multi-source, multi-target problem, and adding s and t with zero-cost connections reduces it to the standard form. Second, the connectivity constraint (2.1) need not be enforced separately: it is fully encoded in the edge set.

2.3 The Shortest-Path Formulation

2.3.1 Optimal Substructure and the Recurrence

The following theorem is the mathematical heart of seam carving. It shows that the well-known dynamic-programming table is precisely the vector of shortest-path distances from the source.

[Recurrence] Let $M(i, j) = \delta(s, (i, j))$. Then

$$M(0, j) = e(0, j), \tag{2.7}$$

$$M(i, j) = e(i, j) + \min_{\substack{d \in \{-1, 0, 1\} \\ 0 \leq j+d < W}} M(i-1, j+d) \quad (i \geq 1), \tag{2.8}$$

and $\delta(s, t) = \min_j M(H-1, j)$. The only path to $(0, j)$ is the source edge, of weight $e(0, j)$, which gives (2.7). Let $i \geq 1$. Every s - (i, j) path ends with an edge $((i-1, j'), (i, j))$ for some admissible $j' = j + d$, and its prefix is an s - $(i-1, j')$ path. Hence its weight is at least $M(i-1, j') + e(i, j)$. Conversely, appending this edge to a *shortest* s - $(i-1, j')$ path yields a path of weight exactly $M(i-1, j') + e(i, j)$. Minimising over the admissible j' gives (2.8). The last claim holds because the sink edges have weight zero.

The argument is an instance of the *optimal-substructure* property of shortest paths (every subpath of a shortest path is itself a shortest path [3]) and of Bellman’s principle of optimality [2]. It explains why a locally computed table suffices to find a globally optimal seam.

2.3.2 Algorithms

The general algorithm for single-source shortest paths in a DAG processes the vertices in topological order and *relaxes* each outgoing edge exactly once [3].

[h] [1] $d[v] \leftarrow \infty$ and $\pi[v] \leftarrow \text{nil}$ for all $v \in V$; $d[s] \leftarrow 0$ each vertex u **in topological order** each edge $(u, v) \in E$ $d[u] + w(u, v) < d[v]$ relaxation of (u, v) $d[v] \leftarrow d[u] + w(u, v)$; $\pi[v] \leftarrow u$

When vertex u is processed, all of its in-neighbours precede it in the topological order and have already been processed, so $d[u] = \delta(s, u)$ is final. The predecessor pointers π form a *shortest-path tree* rooted at s . On the seam graph, the row-by-row order of Lemma 2.2.3 is a topological order, and Algorithm 2.3.2 specialises to the following procedure, which evaluates the recurrence of

Theorem 2.3.1 and recovers the seam by backtracking along π .

[h] [1] Energy map $e \in R_{\geq 0}^{H \times W}$ Minimum-energy seam $\sigma^* = (j_0^*, \dots, j_{H-1}^*)$ and its cost $M(0, j) \leftarrow e(0, j)$ for all j relax the source edges $i = 1$ **to** $H - 1$ topological order: row by row $j = 0$ **to** $W - 1$ $\pi(i, j) \leftarrow \arg \min_{j' \in \{j-1, j, j+1\} \cap [0, W-1]} M(i-1, j')$ $M(i, j) \leftarrow e(i, j) + M(i-1, \pi(i, j))$ $j_{H-1}^* \leftarrow \arg \min_j M(H-1, j)$ cheapest sink edge $i = H - 1$ **down to** 1 backtracking from t $j_{i-1}^* \leftarrow \pi(i, j_i^*)$ σ^* and $M(H-1, j_{H-1}^*)$

2.3.3 Correctness and Complexity

Correctness of Algorithm 2.3.2 follows from Theorem 2.3.1 (the table M satisfies the recurrence, hence contains the true distances $\delta(s, \cdot)$), Corollary 2.2.4, and the fact that following the predecessor pointers from the cheapest last-row vertex traces a shortest s - t path. Ties between equally cheap predecessors may be broken arbitrarily; different tie-breaking rules may return different, but equally optimal, seams.

[Complexity] Algorithm 2.3.2 runs in $\Theta(|V| + |E|)$ time. On the seam graph this is $\Theta(HW)$, and the memory requirement is $\Theta(HW)$. Each vertex is processed once and each edge is relaxed once, at constant cost per relaxation. On G_e the topological order is given by the row structure (Lemma 2.2.3), so no sorting is needed, and $|V| + |E| = \Theta(HW)$ by (2.6). The tables M and π have HW entries each.

If only the *cost* of the optimal seam is needed, keeping two rows of M suffices ($O(W)$ memory); recovering the seam itself requires the predecessor table π .

Comparison with general-purpose algorithms. Table 2.1 compares the DAG algorithm to the standard shortest-path algorithms applied to G_e , and to exhaustive search. Bellman–Ford needs $|V| - 1$ passes over all edges, and Dijkstra’s algorithm needs a priority queue; the DAG algorithm needs neither.

Table 2.1: Cost of finding the minimum seam on G_e ($|V| = HW + 2$, $|E| = \Theta(HW)$).

Method	Weights	Time on G_e
Exhaustive enumeration	any	$\Omega(N(H, W))$, exponential in H
Bellman–Ford	any, no negative cycle	$O(V E) = O(H^2W^2)$
Dijkstra (binary heap) [4]	non-negative	$O(E \log V) = O(HW \log HW)$
DAG relaxation	any	$\Theta(HW)$

Recognising that the graph is acyclic removes the priority queue of Dijkstra’s algorithm and the $|V|$ passes of Bellman–Ford. It also has a second advantage: because a DAG has no cycles, and therefore no negative cycles, the DAG algorithm remains correct for *negative* edge weights, whereas Dijkstra’s algorithm does not. This allows regions to be assigned negative energy to attract seams, as used for object removal (Section 2.4).

Parallelism. Any two vertices of the same row are unreachable from each other, since the level strictly increases along every path. The relaxations within a row are therefore mutually independent

and can be executed simultaneously, for instance with vectorised array operations. Only the H rows themselves form a dependency chain, so the sequential depth of the computation is $\Theta(H)$ while the total work remains $\Theta(HW)$.

2.4 Iterated Removal, Extensions and Limitations

Cost of reducing the width by k pixels. Each of the k iterations recomputes the energy map ($\Theta(HW)$), runs Algorithm 2.3.2 ($\Theta(HW)$) and deletes a seam ($\Theta(HW)$ to copy the image). The total cost is $\Theta(kHW)$. Note that the graph G_e itself changes at every iteration, since the energy is recomputed on the narrower image.

Greedy nature. Each seam is a *globally* optimal solution of the single-seam problem on the current image, as proved above. The *sequence* of k seams, however, is chosen greedily: since every removal modifies the image and hence the graph, an optimal first seam need not belong to an optimal set of k seams. The procedure is thus a heuristic for the joint problem, but it works very well in practice, and it costs only $\Theta(kHW)$. For removals in both dimensions, Avidan and Shamir determine the optimal order of horizontal and vertical seams by dynamic programming over a transport map [1].

Variations are changes of weights. The graph model shows that many extensions modify only the weight function w (or the edge set) of G_e , and leave the algorithm untouched:

- **Forward energy.** The energy (2.4) is a *backward* criterion: it measures the importance of the pixels being removed, but ignores the new discontinuities created when the two sides of the seam are joined. Rubinstein et al. [5] propose *forward energy*, where the cost of an edge depends on the pixels that become adjacent, i.e. on the pair (u, v) rather than on v alone. In our framework this is simply another weight function $w(u, v)$ on the same DAG. They also cast seam finding as a minimum s - t cut, a formulation that extends to seam surfaces in video volumes.
- **Protecting and removing objects.** Assigning very large weights to protected regions makes seams avoid them; assigning small or negative weights to a region marked for removal attracts seams into it. The latter is compatible with the DAG algorithm, but not with Dijkstra's.
- **Horizontal seams** are obtained by running the same algorithm on the transposed energy map.

Limitations. The gradient-based energy has no semantic knowledge. Faces and regular structures (facades, fences) can be visibly distorted when seams must cross them. When an image contains little low-energy content, every seam is costly and the quality of the result degrades. These limitations concern the choice of the *energy*, not the search for the seam: the shortest-path machinery of this chapter remains valid for any non-negative (or, for the DAG algorithm, arbitrary real) weights.

Summary. The key insight of this chapter is that an image induces a layered DAG on its pixels; that vertical seams are exactly its source-to-sink paths (Lemma 2.2.4); and that the minimum-energy seam is therefore a shortest path, computable in $\Theta(HW)$ time by relaxing edges in the topological order that the row structure already provides (Theorems 2.3.1 and 2.3.3). The dynamic-programming table used in practice is the vector of shortest-path distances from the source, and the seam is read off the shortest-path tree by backtracking.

Chapter 3

Implementation

This chapter turns the shortest-path formulation of Chapter 2 into a working program. It describes the data structures, the dynamic-programming (DP) algorithm and its vectorised implementation, the iterated removal of seams, a complete worked example, and the checks used to validate correctness and running time. The program is written in Python with NumPy for array computations and scikit-image [7] for the image operators; the graph library NetworkX is used *only* to validate the results against general-purpose algorithms (Section 3.8).

3.1 Overview

Reducing the width of an image by one pixel consists of four stages, which are repeated k times to remove k seams:

1. compute the energy map e of the current image (Section 2.1.2);
2. fill the DP table M and the predecessor table π (Section 3.3);
3. recover the minimum seam by backtracking (Section 3.3.4);
4. delete the seam from the image (Section 3.5).

Table 3.1 lists the corresponding functions and their asymptotic cost for an $H \times W$ image (analysed in Section 3.7).

Table 3.1: Components of the implementation and their cost per seam.

Stage	Function	Time
Energy map	<code>compute_energy</code>	$\Theta(HW)$
DP table and predecessors	<code>cumulative_energy</code>	$\Theta(HW)$
Backtracking	<code>backtrack</code>	$\Theta(H)$
Seam removal	<code>remove_seam</code>	$\Theta(HW)$
Removal of k seams	<code>carve</code>	$\Theta(kHW)$

3.2 Problem Formulation

The image is modelled as the seam graph G_e of Section 2.2.2, a directed acyclic graph (DAG) with one vertex per pixel and a virtual source s and sink t . By Lemma 2.2.4, finding a minimum-cost path from any pixel in the first row to any pixel in the last row is the classic *single-source shortest path* (SSSP) problem from s to t in G_e .

The image is first converted into an energy map, where each pixel is assigned a cost representing its visual importance (here, the Sobel gradient magnitude (2.4)). From a pixel (i, j) , transitions are allowed only to the three pixels of the next row:

- $(i + 1, j - 1)$,
- $(i + 1, j)$,
- $(i + 1, j + 1)$,

where targets outside the image are omitted, so that pixels in the first and last column have only two successors. Because every transition increases the row index by one, the graph is acyclic and the row-by-row order is a topological order (Lemma 2.2.3).

Implicit graph. The graph is never built as an explicit adjacency structure. The successors of (i, j) are obtained by index arithmetic, so the whole graph is represented by the energy array itself, of size HW . This is possible because the edge set has the regular structure of Section 2.2.2, and it keeps both memory and running time linear in the number of pixels.

3.3 Dynamic Programming Approach

By Theorem 2.3.1, the shortest-path distances from the source satisfy a simple recurrence over the rows. The implementation evaluates this recurrence in topological order and stores the intermediate results in two tables.

3.3.1 Dynamic Programming Table

A DP table M of the same dimensions as the energy map is constructed (named `dp` in the code). Each entry

$$M(i, j)$$

represents the minimum cumulative energy of any seam that reaches pixel (i, j) from the top row, including the energy of (i, j) itself. In graph terms, $M(i, j) = \delta(s, (i, j))$.

A second table π of the same size (named `parent`) stores, for each pixel, the column of the predecessor in the row above that realises this minimum. The predecessor pointers form the shortest-path tree rooted at s (Section 2.3) and are what makes the seam recoverable.

3.3.2 Initialization

The first row of the DP table is initialized directly with the energy values of the first row of the image, since a seam can start at any column:

$$M(0, j) = e(0, j) \quad \text{for all } j.$$

All remaining entries are initialized to infinity,

$$M(i, j) = \infty \quad \text{for } i \geq 1,$$

so that an entry that has not been computed yet can never be selected as a minimum. The predecessor table is initialised with the placeholder -1 .

3.3.3 Recursion Relation

For each pixel (i, j) , starting from the second row, let $J(j) = \{j-1, j, j+1\} \cap \{0, \dots, W-1\}$ denote the set of valid columns in the previous row. The DP value and the predecessor are

$$M(i, j) = e(i, j) + \min_{j' \in J(j)} M(i-1, j'), \quad (3.1)$$

$$\pi(i, j) = \arg \min_{j' \in J(j)} M(i-1, j'). \quad (3.2)$$

Written out for an interior pixel, this is

$$M(i, j) = e(i, j) + \min \begin{cases} M(i-1, j-1), \\ M(i-1, j), \\ M(i-1, j+1). \end{cases}$$

This is exactly the relaxation of the three edges entering (i, j) in Algorithm 2.3.2, and the correctness of the whole procedure follows from Theorem 2.3.1. When several predecessors attain the minimum, we select the one with the smallest column index. This makes the algorithm deterministic; any other rule would return an equally optimal, but possibly different, seam.

3.3.4 Seam Extraction (Backtracking)

Once the DP table is fully computed, the minimum value in the last row corresponds to the end of the optimal seam (the cheapest edge into the sink t), and its value is the seam cost $\delta(s, t)$. Using the predecessor table, the seam is reconstructed by backtracking from the last row to the first row, retrieving the column index at each step:

$$j_{H-1}^* = \arg \min_j M(H-1, j), \quad j_{i-1}^* = \pi(i, j_i^*) \quad (i = H-1, \dots, 1).$$

Backtracking visits one pixel per row and therefore costs only $\Theta(H)$.

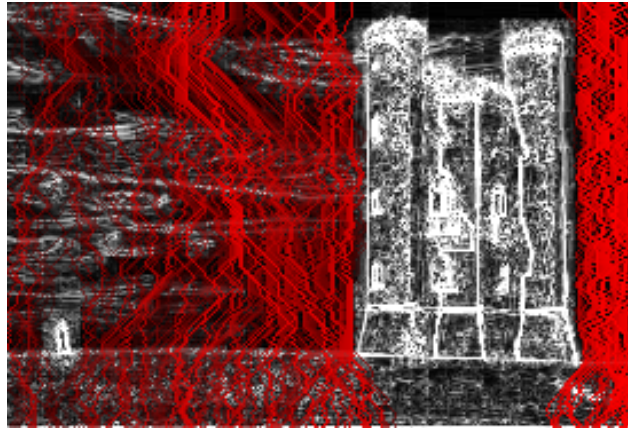


Figure 3.1: Identifying seams of the image.

3.4 Vectorised Implementation

Listing 3.1 shows the core of the implementation. A direct transcription of Algorithm 2.3.2 would use two nested Python loops over the HW pixels, which is slow in an interpreted language. Instead, the code processes a *whole row* at a time with array operations, so that the Python interpreter executes only H loop iterations.

Listing 3.1: Energy map and DP table (vectorised over rows).

```

1 import numpy as np
2 from skimage import color, filters
3
4 def compute_energy(image):
5     gray = color.rgb2gray(image) # luminance in [0, 1]
6     g_y = filters.sobel_h(gray) # vertical derivative
7     g_x = filters.sobel_v(gray) # horizontal derivative
8     return np.hypot(g_x, g_y) # sqrt(g_x^2 + g_y^2)
9
10 def cumulative_energy(e):
11     """Return the DP table (dp) and the predecessor table (parent)."""
12     H, W = e.shape
13     dp = np.full((H, W), np.inf)
14     parent = np.full((H, W), -1, dtype=int)
15     cols = np.arange(W)
16     dp[0] = e[0] # a seam may start in any column
17     for i in range(1, H): # topological order: row by row
18         prev = np.concatenate([np.inf, dp[i - 1], np.inf])
19         # candidates through the left, upper and right predecessor of every pixel
20         cand = np.stack((prev[:-2], prev[1:-1], prev[2:]))
21         k = cand.argmin(axis=0) # 0: left, 1: up, 2: right
22         dp[i] = e[i] + cand[k, cols]
23         parent[i] = cols + k - 1
24     return dp, parent

```

Three implementation details deserve explanation.

- **Border handling.** The previous row is padded with $+\infty$ on both sides. A missing neighbour at the left or right border then simply never wins the minimum, so no special case is needed for the first and last columns. This corresponds to the omitted edges of the graph.
- **Tie-breaking.** The three candidates are stacked in the order left, up, right, and `argmin` returns the first minimum. Ties are thus resolved towards the smallest column index, matching the rule of Section 3.3.
- **Why row-parallelism is legitimate.** Two vertices of the same row are unreachable from each other, because the row index strictly increases along every path (Lemma 2.2.3). The entries $M(i, \cdot)$ therefore depend only on row $i - 1$ and never on each other, and can be computed simultaneously. Only the H rows themselves form a chain of dependencies.

The seam is then recovered by Listing 3.2, a direct implementation of the backtracking rule of Section 3.3.4. The seam is represented as an integer array whose i -th entry is the column of the seam in row i .

Listing 3.2: Backtracking the minimum seam.

```
1 def backtrack(dp, parent):
2     H = dp.shape[0]
3     seam = np.empty(H, dtype=int)
4     seam[-1] = dp[-1].argmin() # cheapest edge into the sink t
5     for i in range(H - 1, 0, -1):
6         seam[i - 1] = parent[i, seam[i]] # follow the shortest-path tree upwards
7     return seam
```

3.5 Seam Removal and Iterated Carving

3.5.1 Seam Removal

After identifying the seam, it is removed by deleting one pixel per row and shifting the remaining pixels to the left. The image width is therefore reduced by one. Listing 3.3 implements this with a Boolean mask: the mask is `True` everywhere except at the seam, and indexing the image with it drops exactly one pixel per row.

To remove multiple seams, the process is repeated. After each removal, the energy map and DP table are recomputed to reflect the updated image. Recomputation is necessary because the removal makes formerly non-adjacent pixels neighbours, which changes the gradients and hence the edge weights of the seam graph.

3.5.2 Tracking Seams in Original Coordinates

A seam found in the t -th iteration is expressed in the coordinates of the *current*, already narrowed image. Its column indices do not correspond to columns of the original image, and drawing them on the original would misplace every seam after the first. To report all seams in the coordinates of the original image, the implementation maintains an *index map* `origin`, where `origin[i, j]` is the original column of the pixel that currently sits at (i, j) . The same removal is applied to this map after every iteration, and the original columns of the seam are read from it before it is updated.

Listing 3.3: Seam removal and iterated carving with tracking of original coordinates.

```
1 def remove_seam(arr, seam):
2     H, W = arr.shape[:2]
3     keep = np.ones((H, W), dtype=bool)
4     keep[np.arange(H), seam] = False # one pixel per row is dropped
5     return arr[keep].reshape((H, W - 1) + arr.shape[2:])
6
7 def carve(image, k):
8     H, W = image.shape[:2]
9     rows = np.arange(H)
10    origin = np.tile(np.arange(W), (H, 1)) # origin[i, j]: original column of pixel (i,
        j)
11    seams = []
12    for _ in range(k):
13        dp, parent = cumulative_energy(compute_energy(image))
14        seam = backtrack(dp, parent)
15        seams.append(origin[rows, seam]) # seam in ORIGINAL coordinates
16        image = remove_seam(image, seam)
17        origin = remove_seam(origin, seam)
18    return image, np.array(seams) # seams[t, i]: column of seam t in row i
```

Two properties of the resulting seams are worth stating. First, the k seams are pairwise *disjoint* in the original image, since every pixel is removed at most once. Second, they need *not* be 8-connected there: pixels removed by earlier seams lie between the pixels of a later seam. Connectivity holds only in the image in which the seam was found. Figure 3.2 shows the tracked seams on the original image.

3.6 Step-by-Step Example

Consider a small 4×5 image with the following energy values:

5	3	2	4	6
7	2	1	3	5
3	5	2	1	4
4	2	3	2	5

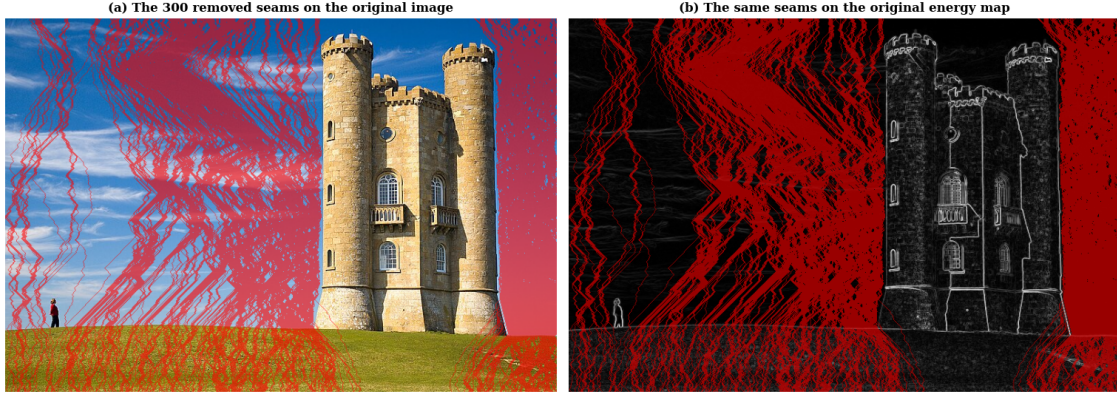


Figure 3.2: The removed seams, drawn in the coordinates of the original image (left) and on its energy map (right). The seams run through low-energy regions and avoid the high-energy structures.

where $e(i, j)$ is the Sobel gradient magnitude at pixel (i, j) . Rows and columns are numbered from 0.

Step 1: Initialize the First Row

The DP table is initialized with the energy values of the first row:

$$M(0, \cdot) = \begin{bmatrix} 5 & 3 & 2 & 4 & 6 \end{bmatrix}.$$

Step 2: Process Row 1

For each pixel in row 1 we compute

$$M(1, j) = e(1, j) + \min (M(0, j - 1), M(0, j), M(0, j + 1)),$$

restricted to valid columns. The detailed calculations are:

- **Pixel (1,0):** can come from (0, 0) or (0, 1);

$$M(1, 0) = 7 + \min(5, 3) = 7 + 3 = \mathbf{10}, \quad \pi(1, 0) = 1.$$

- **Pixel (1,1):** can come from (0, 0), (0, 1) or (0, 2);

$$M(1, 1) = 2 + \min(5, 3, 2) = 2 + 2 = \mathbf{4}, \quad \pi(1, 1) = 2.$$

- **Pixel (1,2):** can come from (0, 1), (0, 2) or (0, 3);

$$M(1, 2) = 1 + \min(3, 2, 4) = 1 + 2 = \mathbf{3}, \quad \pi(1, 2) = 2.$$

- **Pixel (1,3):** can come from (0, 2), (0, 3) or (0, 4);

$$M(1, 3) = 3 + \min(2, 4, 6) = 3 + 2 = \mathbf{5}, \quad \pi(1, 3) = 2.$$

- **Pixel (1,4):** can come from (0, 3) or (0, 4);

$$M(1, 4) = 5 + \min(4, 6) = 5 + 4 = \mathbf{9}, \quad \pi(1, 4) = 3.$$

Result after processing row 1:

$$M(1, \cdot) = \begin{bmatrix} 10 & 4 & 3 & 5 & 9 \end{bmatrix}.$$

Steps 3 and 4: Process Rows 2 and 3

The same recurrence is applied to the remaining rows. Table 3.2 lists all candidates.

Table 3.2: Computation of rows 2 and 3. The column “candidates” lists $M(i-1, j')$ for the valid predecessors $j' \in J(j)$ in increasing order of j' .

Pixel	$e(i, j)$	Candidates	Min	$M(i, j)$	$\pi(i, j)$
(2, 0)	3	10, 4	4	$3 + 4 = 7$	1
(2, 1)	5	10, 4, 3	3	$5 + 3 = 8$	2
(2, 2)	2	4, 3, 5	3	$2 + 3 = 5$	2
(2, 3)	1	3, 5, 9	3	$1 + 3 = 4$	2
(2, 4)	4	5, 9	5	$4 + 5 = 9$	3
(3, 0)	4	7, 8	7	$4 + 7 = 11$	0
(3, 1)	2	7, 8, 5	5	$2 + 5 = 7$	2
(3, 2)	3	8, 5, 4	4	$3 + 4 = 7$	3
(3, 3)	2	5, 4, 9	4	$2 + 4 = 6$	3
(3, 4)	5	4, 9	4	$5 + 4 = 9$	3

Hence

$$M(2, \cdot) = \begin{bmatrix} 7 & 8 & 5 & 4 & 9 \end{bmatrix}, \quad M(3, \cdot) = \begin{bmatrix} 11 & 7 & 7 & 6 & 9 \end{bmatrix}.$$

Complete DP and Predecessor Tables

The final DP table M (minimum cumulative energy to reach each pixel) and the predecessor table π are shown below. The entries of M on the optimal seam, determined in Step 6, are highlighted.

DP table M					Predecessor table π				
5	3	2	4	6	—	—	—	—	—
10	4	3	5	9	1	2	2	2	3
7	8	5	4	9	1	2	2	2	3
11	7	7	6	9	0	2	3	3	3

Step 5: Find the Minimum Seam End

The minimum value in the last row is

$$\min M(3, \cdot) = \min(11, 7, 7, 6, 9) = \mathbf{6} \quad \text{at column } j = 3.$$

This is the cost $\delta(s, t)$ of the optimal seam.

Step 6: Backtrack to Reconstruct the Seam

Starting from pixel $(3, 3)$ with cost 6, we follow the predecessor pointers upwards:

$$\begin{aligned} (3, 3) &\xrightarrow{\pi(3,3)=3} (2, 3) \\ (2, 3) &\xrightarrow{\pi(2,3)=2} (1, 2) \\ (1, 2) &\xrightarrow{\pi(1,2)=2} (0, 2). \end{aligned}$$

The optimal seam is therefore $\sigma^* = \{(0, 2), (1, 2), (2, 3), (3, 3)\}$, i.e. the column sequence $(2, 2, 3, 3)$, with total energy $2 + 1 + 1 + 2 = 6$. Note that the DP values along the seam, $2, 3, 4, 6$, are the running sums of the energies collected so far.

Visual Representation of the Optimal Seam

5	3	2	4	6
7	2	1	3	5
3	5	2	1	4
4	2	3	2	5

The highlighted cells show the optimal seam. The sum of the energy values along this path is $2 + 1 + 1 + 2 = 6$. This is the minimum-energy vertical seam of the image: an exhaustive enumeration of all 95 vertical seams of a 4×5 grid confirms that no seam has a lower cost, and that this one is the unique minimiser.

Step 7: Remove the Seam

Deleting the highlighted pixels and closing the gaps gives a 4×4 image (with the energies as they were before removal):

5	3	4	6
7	2	3	5
3	5	2	4
4	2	3	5

In the full algorithm the energy map of this narrower image is recomputed from the pixel values before the next seam is searched (Section 3.5); the numbers above are only the old energies of the surviving pixels.

3.7 Complexity of the Implementation

Time. For one seam, the energy map ($\Theta(HW)$: a constant number of operations per pixel), the DP table ($\Theta(HW)$ by Theorem 2.3.3), the backtracking ($\Theta(H)$) and the removal ($\Theta(HW)$: one copy of the image) are all linear in the number of pixels. Removing k seams costs

$$T(k) = \sum_{t=0}^{k-1} \Theta(H(W-t)) = \Theta(kHW),$$

because the width shrinks in every iteration but remains $\Theta(W)$ as long as $k \leq W/2$. For k proportional to W this is quadratic in the image width.

Memory. The DP table and the predecessor table each contain HW entries, in addition to the image and its energy map. With 64-bit entries, each table occupies about 3.5 MB for an 800×542 image. Since only the previous row is needed to compute the costs, the memory of the DP table could be reduced to $O(W)$ if the seam did not have to be recovered; the predecessor table, however, is required for backtracking.

Vectorisation. The row-wise vectorisation of Section 3.4 does not change the operation count, which remains $\Theta(HW)$. It removes the interpreter overhead of the HW inner-loop iterations and executes the arithmetic in compiled array code, leaving only H interpreted iterations. Figure 3.3 compares the running time of one shortest-path computation for three implementations on random energy maps of increasing size: the vectorised DP, a literal element-by-element transcription of Algorithm 2.3.2, and Dijkstra’s algorithm on an explicitly constructed NetworkX graph. In our measurements none of the three grows faster than roughly linearly in the number of pixels over the tested range, and the vectorised version is faster by a large constant factor. The logarithmic factor of Dijkstra’s algorithm is too small to be resolved in this range, so the figure supports the claim that the DAG algorithm is at least as fast as Dijkstra while being simpler.

The energy map does not need to be recomputed in its entirety after a removal. Only pixels whose 3×3 neighbourhood contains the deleted seam change, i.e. a band of constant width around the seam, so this stage could be updated in $O(H)$ time. The DP table, on the other hand, depends on the whole energy map and would still cost $\Theta(HW)$, so the asymptotic cost per seam is unchanged. We therefore recompute the energy map in full, which is simpler and less error-prone.

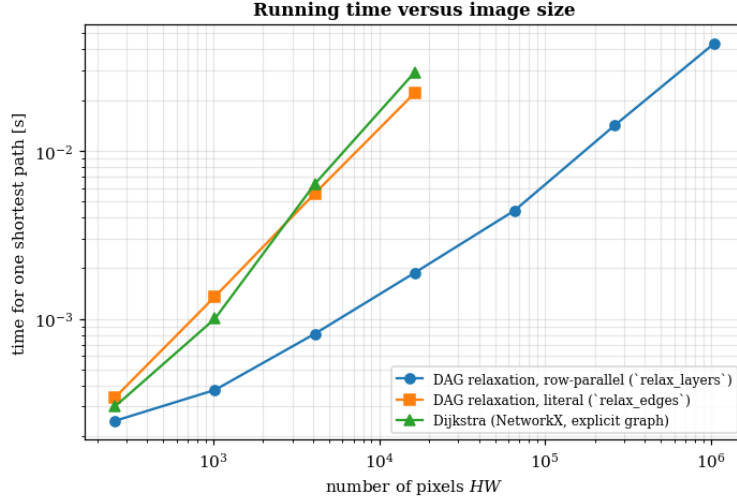


Figure 3.3: Running time of one minimum-seam computation versus the number of pixels HW (log–log scale), for the vectorised DP, a literal loop implementation of Algorithm 2.3.2, and Dijkstra’s algorithm (NetworkX). Absolute times depend on the machine.

3.8 Verification

The correctness of the implementation was checked at three independent levels.

1. Exhaustive enumeration. On small instances, the minimum seam cost was compared with the minimum over *all* seams, enumerated directly from Definition 2.1.1, without any graph theory. We used 100 random energy maps of size 6×5 , half with continuous values and half with small integer values (which create many ties). Both the vectorised DP and a literal loop implementation returned a valid seam whose cost equals the enumerated optimum in all 100 instances. The number of seams of a 6×5 grid is small enough for this to be feasible, whereas for realistic sizes it is astronomically large (Section 2.1.1).

2. Cross-validation with general-purpose graph algorithms. For a random 40×60 energy map, the seam graph G_e was built explicitly with NetworkX ($|V| = 2402$, $|E| = 7062$, matching $HW + 2$ and $(H - 1)(3W - 2) + 2W$; the graph was confirmed to be acyclic). The distances $\delta(s, v)$ to *every* vertex were computed with Dijkstra’s algorithm, with Bellman–Ford, and with a generic topological-order relaxation that knows nothing about the grid. All three agree with the DP table M in every entry, not merely in the final seam cost. The vectorised and the literal implementation also produce identical predecessor tables, because both break ties towards the smallest column.

3. Invariants of iterated removal. After removing k seams with the code of Listing 3.3, we checked that (i) the seams, expressed in original coordinates, are pairwise disjoint, so that exactly $k \cdot H$ distinct pixels are removed; and (ii) deleting exactly these pixels from the original image reproduces the carved image bit for bit. Check (ii) validates the index map `origin` and would fail if seams were recorded in the coordinates of the narrowed image.

3.9 Implementation Notes

- **Data types.** The energy and DP tables use 64-bit floating point numbers, which allow the value $+\infty$ for padding and uninitialised entries and avoid overflow of cumulative sums. Images are kept in their original integer type; only the energy computation converts them to floating point.
- **Image borders.** The Sobel filter of scikit-image reflects the image at its borders, so border pixels receive meaningful energies without special treatment.
- **Horizontal seams.** A horizontal seam is obtained by transposing the image (and, equivalently, the energy map), applying the same code, and transposing back.
- **Guard on the number of seams.** At least one column must remain, so the number of seams to be removed is required to be smaller than the width.
- **Greedy strategy.** Each seam is optimal for the current image, but the sequence of k seams is chosen greedily (Section 2.4). The implementation makes no attempt to optimise the set of seams jointly.

Chapter 4

Conclusion

In this project, we studied and implemented the content-aware image resizing technique known as seam carving, using concepts from graph theory and dynamic programming. By modeling the image as a weighted directed acyclic graph, where each pixel represents a node and edges encode admissible seam transitions, the problem of identifying an optimal seam was reduced to a shortest path computation.

An energy map was constructed to quantify the visual importance of each pixel. This map was obtained by converting the image to grayscale and applying the Sobel operator to compute gradient magnitudes, allowing edges and textured regions to be preserved during resizing. Dynamic programming was then employed to efficiently compute the minimum-energy vertical seams by propagating cumulative energy values from the top row to the bottom row.

The proposed approach enables the progressive removal of low-energy seams, leading to image resizing that minimizes visual distortion compared to uniform scaling. Experimental results demonstrate that the method successfully preserves salient structures while reducing image dimensions.

Overall, this project highlights how classical graph algorithms and optimization techniques can be effectively applied to practical problems in image processing, bridging the gap between theoretical concepts and real-world applications.